

Modul 5 Eksplorasi Proses

Tujuan Praktikum

1. Praktikan memahami PCB pada Xinu.
2. Praktikan mampu memanfaatkan informasi PCB pada program yang dibuat.

5.1 Proses

1. Sistem operasi menyimpan semua informasi mengenai semua proses yang berjalan pada struktur data yang disebut sebagai *process table*. Sebuah proses direpresentasikan sebagai sebuah entri dalam *process table* tersebut. Entri pada *process table* akan dibuat pada saat proses diciptakan dan entri pada *process table* akan dihapus pada saat proses diterminasi.
2. Beberapa source code utama untuk menangani proses adalah:
 - a. `./include/proses.h` yang berisi konfigurasi setiap proses pada Xinu.
 - b. `./system/create.c` untuk membuat proses.
 - c. `./system/kill.c` untuk terminasi proses.
 - d. `./system/resume.c` untuk resume proses.
3. Pada Xinu, implementasi *process table* menggunakan global array bernama `proctab[]`. Deklarasi `proctab[]` dapat dilihat pada `./include/process.h`. Pada source code `process.h` menggunakan keyword `extern` yang membuat variable array `proctab[]` bersifat global (yaitu dapat diakses oleh fungsi apapun). Berikut adalah ilustrasi array `proctab[]`:

0	1	2	3	4	5	6
Struct procent{...A...}	Struct procent{...B...}	Struct procent{...C...}	Struct procent{}	Struct procent{}	Struct procent{}	Struct procent{}

Pada ilustrasi di atas, array `proctab[]` berindex ke 0 akan berisi struct `procent{...A...}`. Isi dari struct `procent` dapat dilihat pada `./include/process.h`

4. Isi dari array `proctab[]` adalah **process control block (PCB) suatu proses**. Pada Xinu, isi dari setiap array `proctab[]` berupa **struct procent** (`./include/process.h`). Dengan kata lain **struct procent adalah PCB**. Berikut adalah struct `procent`:

```

41  /* Definition of the process table (multiple of 32 bits) */
42
43  struct procent {          /* Entry in the process table      */
44      uint16 prstate;      /* Process state: PR_CURR, etc. */
45      pri16 prprio;        /* Process priority          */
46      char *prstkptr;      /* Saved stack pointer       */
47      char *prstkbase;     /* Base of run time stack    */
48      uint32 prstklen;     /* Stack length in bytes     */
49      char prname[PNMLEN]; /* Process name              */
50      sid32 prsem;         /* Semaphore on which process waits */
51      pid32 prparent;      /* ID of the creating process */
52      umsg32 prmsg;        /* Message sent to this process */
53      bool8 prhasmsg;      /* Nonzero iff msg is valid   */
54      int16 prdesc[NDESC]; /* Device descriptors for process */
55  };

```

Gambar 5-1 Struktur Procent

Jadi untuk mengelola suatu proses, Xinu cukup mengubah struktur data **struct procent** pada proses yang ingin dikelola. Sebagai contoh, jika ingin mengubah nama proses maka kernel Xinu cukup mengupdate variabel **char prname**. Bagaimana Xinu mengelola suatu proses? Proses diidentifikasi menggunakan ID (suatu angka unik, mirip dengan nim mahasiswa).

5. Xinu menggunakan implicit data structure yaitu dengan tidak secara eksplisit membuat ID sebagai entri pada struct procent (tidak ada variabel ID pada **struct procent** atau ID pada proses). Alih-alih, **process ID merupakan indeks pada array proctab[]**. Atau dengan kata lain sebuah proses direferensi hanya oleh indeks pada array proctab[]. Contoh: jika Xinu mengakses proctab[3] artinya Xinu sedang mengakses/mengelola proses dengan pid = 3 dan kemudian Xinu dapat memanipulasi misalkan nilai prioritas proses tersebut melalui struct procent yang ada.

Contoh kode:

```

struct procent*prptr;          /* pointer to process      */

prptr = &proctab[3];          /*mengakses array indeks ke-3 (ID = 3) */

printf("%s", prptr->prname);   /* print nama proses dengan ID = 3 *

```